

Sparse memory. Procedural skills. Recurrent geometric reasoning.

A CPU/ARM-first architecture built from exact Engram memory, a broad frozen definition bank, composed skill LoRAs, high-dimensional expert coordinates, signed HarMax geometry, LNGram latent lookup, and two recurrent physical blocks.



TECHNICAL ARCHITECTURE BRIEF

REVISION 11.1 | 25 August 2026

Contents

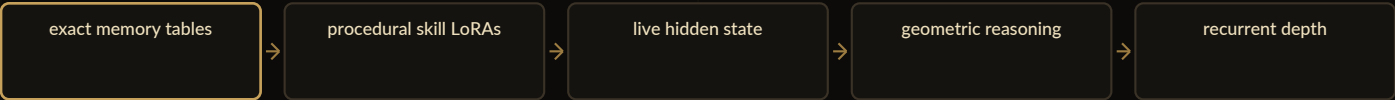
Memory first. Then procedural skill. Then the reasoning engine that uses both.

01	Architecture summary	3
02	Memory - what is stored	4
03	Phrase Engrams - exact lookup tables	5
04	Definition memory - broad English sense bank	6
05	Memory runtime - exact retrieval + JTD	7
06	Skill LoRAs - procedural movement	8
07	How the skill LoRAs will be made	9
08	Shared / private / subspace geometry	10
09	Reasoning engine - live hidden state	11
10	Experts + branches	12
11	LNGram - latent trajectory lookup	13
12	Harmonic Loss / HarMax	14
13	DeepLoop - two recurrent blocks	15
14	DeepLoop - scaling rule	16
15	Research basis + evidence boundary	17

Dendritron in one page

Three layers of capacity: stored knowledge, learned procedure, and recurrent reasoning.

Dendritron separates three jobs that are normally mixed inside a deep Transformer. Memory stores knowledge that can be addressed directly. Skill LoRAs store reusable procedural movement. The reasoning engine carries the current problem through a live 2,048-dimensional hidden state and repeatedly applies experts, branches, latent lookup, geometric evidence, and two reused physical blocks.



PERSISTENT KNOWLEDGE	ACTIVE / LEARNED COMPUTE
500k exact bigram Engrams 500k exact trigram Engrams 1.53M frozen definition senses fitted JTD alignment assets	operational skill LoRAs oblique shared structure high-dimensional expert coordinates two recurrent physical blocks

Design target

The architecture is intended to move capacity out of repeated dense computation and into sparse lookup plus conditional computation, while keeping the active reasoning state small enough for CPU/ARM execution. The following pages separate what is already built from what remains a training or integration hypothesis.

What is stored

The memory system is a set of frozen, addressable tables. This section describes the stored knowledge before describing the reasoning machinery that consumes it.

Phrase memory

Two exact Engram lookup tables store recurring two-word and three-word phrases. A phrase is the key. Its row contains two frozen 2,048D hidden-state representations: one from Qwen Layer 8 and one from Qwen Layer 24.

Definition memory

A separate one-word dictionary bank stores one frozen 2,048D Layer-2 hidden state for every retained word sense, together with its exact definition, source identity, sense identity, and ordered definition-word links.

Why keep them separate

The phrase tables preserve narrow multi-word context. The definition bank preserves broad single-word sense structure and polysemy. They are separate assets with different address spaces and different roles; neither table is the live reasoning state.

PHRASE ENGRAMS exact UTF-8 2g / 3g keys 1,000,000 total rows L8 [2048] BF16 L24 [2048] BF16	DEFINITION BANK 1,532,746 sense rows exact source definitions L2 [2048] BF16 ordered definition-word links
--	---

The Qwen model used to construct these rows is not part of runtime. Once built, the memory consists of the finished frozen tables and their lookup indexes.

Phrase Engrams: exact lookup tables

DeepSeek-style conditional memory populated with hidden states extracted once from Qwen.

The phrase memory is two independent exact lookup tables. The bigram and trigram banks have separate row numbers, indexes, shards, and manifests.

Table	Rows	Exact key	Frozen row payload
Bigram Engram	500,000	exact two-word phrase	Layer 8 [2048] BF16 + Layer 24 [2048] BF16
Trigram Engram	500,000	exact three-word phrase	Layer 8 [2048] BF16 + Layer 24 [2048] BF16

How the rows were constructed

During the offline build, each isolated phrase was passed through Qwen once. The final non-padding token hidden states from Layers 8 and 24 were written into that phrase's row. After extraction, the rows were frozen.



Runtime

Runtime does not call Qwen and does not reconstruct the phrase representation. The exact phrase key resolves the stored row and the model reads the two frozen hidden-state payloads directly.

```
exact phrase -> exact Engram row -> {L8[2048], L24[2048]}
```

Definition memory: broad English sense coverage

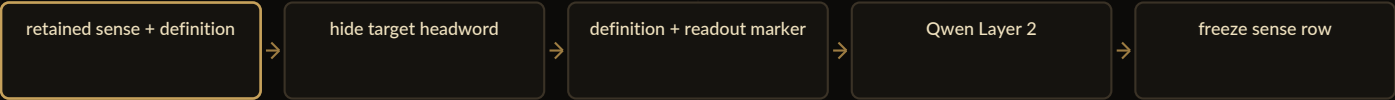
The dictionary was built broadly first. Coverage against the phrase Engrams was checked afterward.

The goal was not to make a dictionary only for words already present in the Engram tables. The source build first assembled a broad English single-word inventory and retained every eligible sense available from the locked lexical sources.

Source	Role
Open English WordNet Plus 2025+	curated general English senses and polysemy
English Wiktionary / Wiktextextract 2026-07-06	broad lexical, scientific, and mathematical vocabulary
MeSH 2026 descriptors	biomedical and life-science terminology
MeSH 2026 supplementary concepts	chemicals, diseases, organisms, protocols, emerging terms

How a definition became a hidden-state row

Each retained sense kept its external word/sense identity, exact source definition, version, and provenance. During Qwen extraction, the target headword itself was withheld from the model input. Qwen saw the definition text and a fixed readout marker; the Layer-2 hidden state at the marker became the frozen 2,048D representation for that sense.



Coverage audit came afterward

After the broad dictionary inventory existed, every unique word appearing in the completed 500k bigram and 500k trigram Engram keys was checked against the dictionary. Any measured gaps were closed with reviewed source-grounded supplements. Engram coverage therefore verifies the dictionary; it did not define the dictionary in the first place.

Memory runtime: exact retrieval + JTD

Only enough machinery to explain how the frozen tables become usable numerical inputs.

Exact surface lookup

3-word Engram -> else 2-word Engram -> else all 1-word dictionary senses

Lookup operates at complete-word boundaries. Phrase keys select one immutable Engram row. When neither phrase table hits, the one-word route returns every matching sense row so polysemy remains explicit.

Joint Transfer Domain

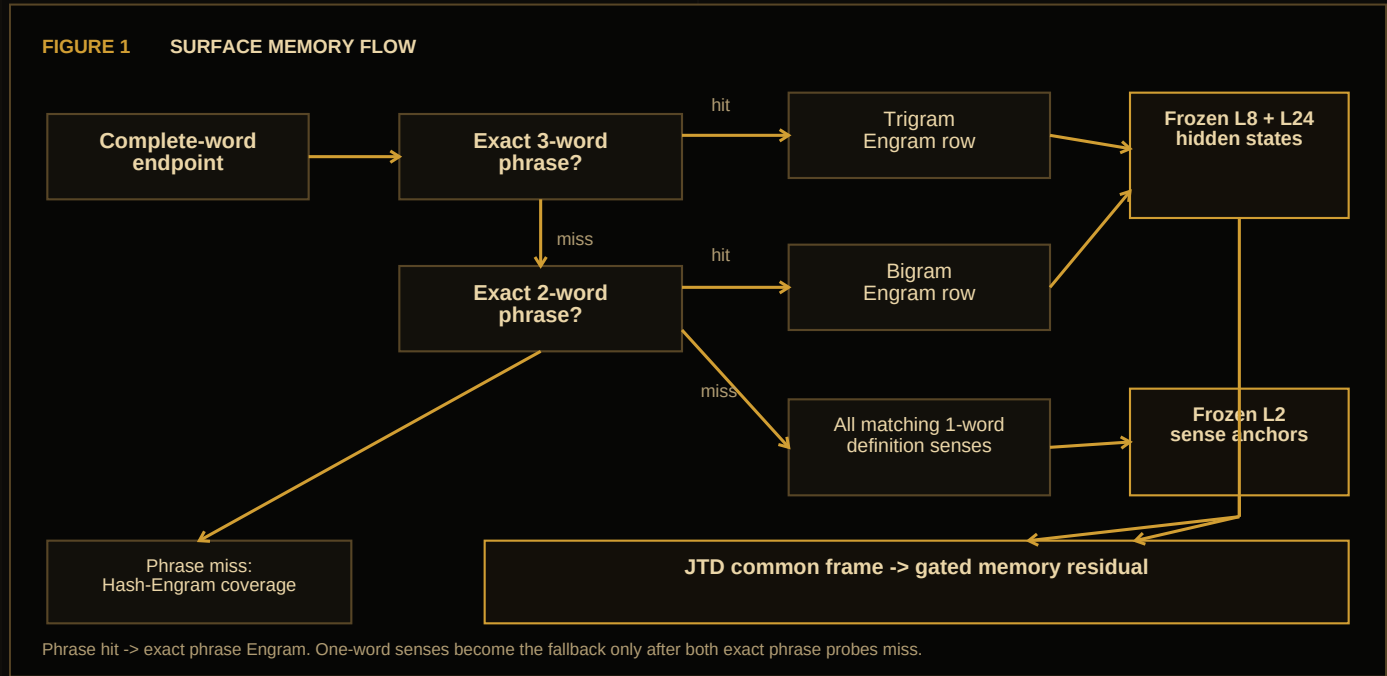
The definition bank's Layer-2 geometry is the common 2,048D reference frame. Separate learned maps align the phrase L8 view, phrase L24 view, and recipient hidden states with that frame without changing the frozen rows:

$$J8(e_{L8}) \ J24(e_{L24}) \ Jh(h) \ definition_{L2} = identity$$

What JTD is not

The integer row IDs are lookup handles, not semantic coordinates. JTD is the continuous alignment layer. Exact lookup chooses the row; JTD makes retrieved continuous values comparable in one frame.

ADDRESSING	CONTINUOUS VALUES
exact phrase text / canonical tuple	L8/L24 phrase hidden states
independent 2g / 3g row spaces	L2 definition hidden states
sense-row handles for words	aligned into common frame
deterministic lookup + provenance	frozen memory values stay fixed



Procedural movement

Skills encode how reasoning proceeds. They are trained separately from answer-bearing expert structure.

The operational skill set is intended to capture reusable moves such as compare, retrieve, reverse, derive, verify, decompose, sequence, and compose. A skill should remain useful when the subject matter and final answer change.

Operational skills stay distinct

The ordinary skill LoRAs are treated as distinct / orthogonal procedural directions so one operation does not collapse into another. A skill receives the complete current hidden state; orthogonality refers to the learned update geometry, not to throwing away input dimensions.

Why this matters

If the adapter is trained to imitate an entire reasoning trace, it can learn the surface sequence - including when the teacher happened to reverse direction - without learning the state-dependent reason for that change. The skill target is therefore the movement of reasoning, not the final answer text.

SKILL	ANSWER STRUCTURE
procedural operation	specialized resolved knowledge
direction of reasoning movement	represented by expert coordinates
reusable across domains	used to verify / support outcomes
learned from successful trajectories	kept separate from skill motion

How the skill LoRAs will be made

The answer verifies the episode. The skill adapter is trained on the reasoning movement that reached it - including corrected changes of direction.

checkpoint -> skill move -> wrong continuation? -> corrected move -> verified answer -> offline update

1. Teach one operation at a time

Build COMPARE, DERIVE, VERIFY, REVERSE, etc. from many unrelated domains. The teacher marks the current reasoning checkpoint and the next reasoning checkpoint that expresses the operation. Native reasoning traces can supplement this, but they are process supervision - not the geometry itself.

2. Freeze everything except the named skill

Base weights, memory tables, experts, other skills, and the shared structure stay fixed. Only the selected operational skill adapter is allowed to change. The final answer is available to the verifier, but answer text does not provide gradient to the skill LoRA.

3. Train the movement, not the wording

For the same current state, measure what activating the skill does to Dendritron's own hidden state and align that movement with the teacher-authored next checkpoint.

```
d_target = stopgrad(h_next - h_current)
d_skill  = h_with_skill - h_without_skill
L_dir    = 1 - cosine(d_skill, d_target)
```

4. Wrong -> corrected is a training pair

If the model first moves from A -> B, fails to reach the needed answer, and later learns that A -> C was the successful continuation, the same checkpoint gives a negative and a positive direction. The corrected episode teaches the change in reasoning; the expert / right answer only tells us which episode succeeded.

5. Commit only verified experience

Save the candidate adapter delta for each episode. Test the operation on held-out domains, paraphrases, and replay cases. Reject or roll back updates that memorize lexical style, damage another skill, or improve token loss without improving the actual procedure.

6. Build shared structure after real skills exist

The first result is a set of ordinary operational skill LoRAs. Keep those skill directions distinct / orthogonal, then stack successful real Delta W updates across skills. SVD/HOSVD finds recurring covariance. That recurring component becomes the oblique shared LoRA; each skill keeps its measured private residual.

WHY THIS TRAINING PATH

Procedural Knowledge Is Not Low-Rank reports that low-rank adapters can preserve the form of a multi-step interaction while missing the state-dependent procedure, and that successful full-finetuning updates are high-rank - especially in MLPs. Stealing Reasoning Traces supports the value of intermediate process supervision. Dendritron therefore uses reasoning traces/checkpoints as supervision, but evaluates the skill through its own hidden-state movement, behavioral success, replay, and measured update spectrum rather than assuming that next-token loss means the procedure was learned.

Shared, private, and subspace geometry

The repeated covariance is shared; the operational skill directions remain distinct.

Start from real successful skill updates

Train and verify operational skill LoRAs first. Stack their successful Dendritron parameter updates and analyze the recurring structure with SVD/HOSVD rather than defining the shared geometry before data exists.

Oblique shared LoRA

The component that repeatedly co-occurs across different skills becomes the shared LoRA. Because it represents covariance, that shared structure is intentionally oblique rather than forced into an orthogonal set that would erase the overlap being measured.

Orthogonal skill LoRAs + private residuals

The normal operational skill directions remain distinct / orthogonal around the shared component. After reconstructing the shared contribution, each skill keeps the remaining private residual; its rank is selected from the measured spectrum rather than fixed in advance.

$$\Delta W_{\text{skill}} = \text{oblique shared contribution} + \text{skill-specific residual}$$

16-32 principled directions

The retained low-dimensional directions are compact coordinates for reusable procedural structure. They are not complete skills. The skills remain full update operators; the 16-32 directions summarize recurring shared structure discovered across them.

The live hidden state carries the current problem

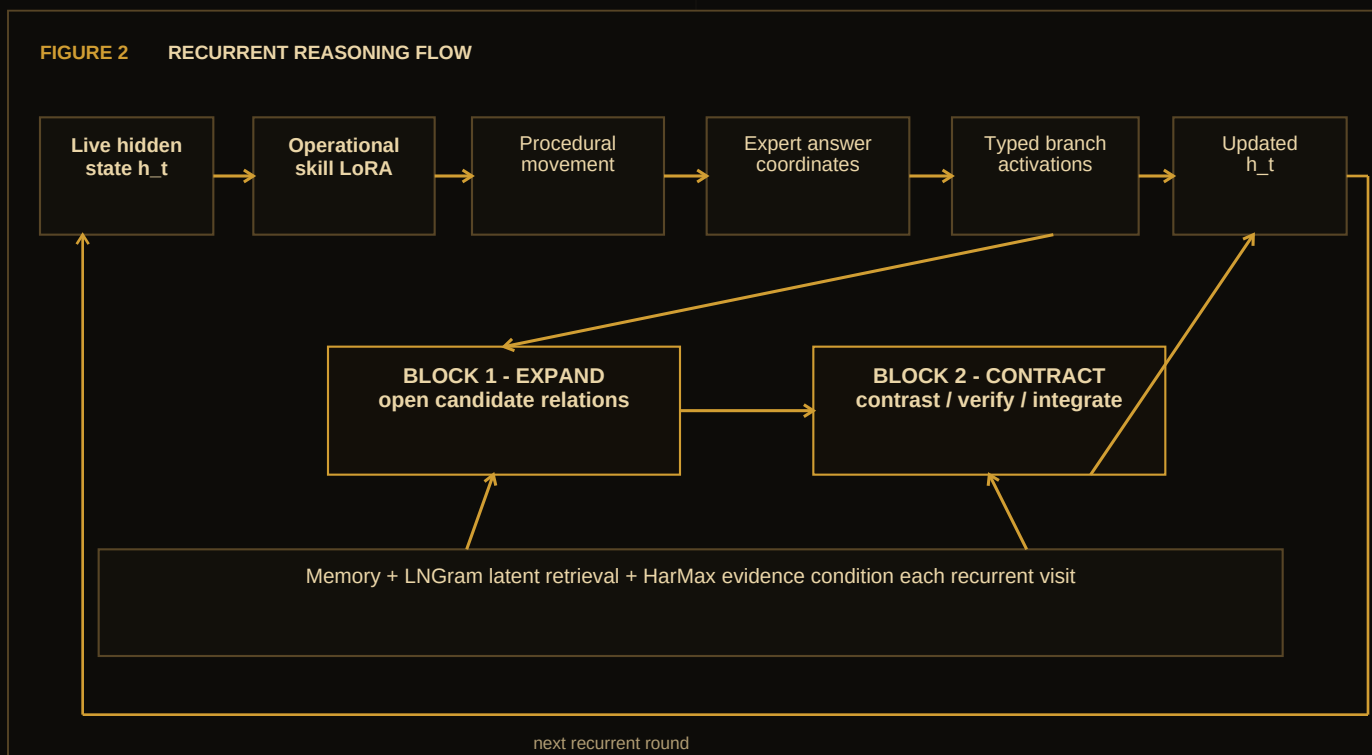
Reasoning begins after memory and skill capacity are defined. The active state is separate from every frozen memory table.

Dendritron's working representation is an evolving 2,048D hidden state, h_t . It carries the current context, intermediate conclusions, active procedural movement, and the effects of retrieved memory through repeated block visits. Frozen Engram and definition rows remain stored values; they never become the persistent working state themselves.

Reasoning capacity

The operational skill LoRAs supply learned movement. Expert coordinates supply high-dimensional specialized answer structure. Branches bind those coordinates to the current state. LN Gram provides exact latent lookup from the state trajectory. HarMax turns bounded evidence into signed geometric movement. DeepLoop reuses two physical blocks to repeat the process.

The order among some nonlinear components remains an experimental integration choice; the architectural ownership of each component is kept separate.



High-dimensional answer coordinates and typed reasoning paths

Expert structure is higher-dimensional than the compact procedural skill geometry.

Expert coordinates

Experts are high-dimensional coordinates formed from verified specialized answer structure that is not well represented by the compact procedural skill basis. They are answer-bearing coordinates: when the needed structure is represented, an expert can support the current reasoning; when it is absent, there is no answer contribution from that expert.

Reverse Gram-Schmidt / stable QR-SVD protection

Expert coordinates are given priority during consolidation. The high-dimensional expert span is anchored first, then the lower-dimensional operational skill geometry is projected or re-orthogonalized around it. The implementation should use stable QR/SVD/null-space projection rather than literal classical Gram-Schmidt.

Branches

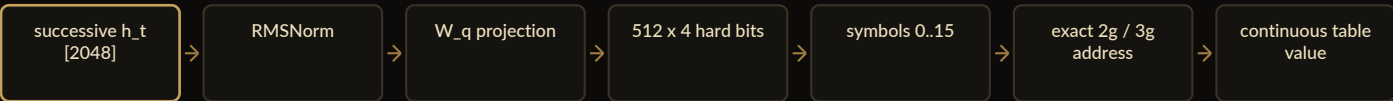
Each expert owns compact branch specifications. Their numerical activations are recreated from the latest live hidden state on every visit. Branches are the typed relationship paths through which current reasoning can bind to expert answer structure.

Operator	Role
Deductive	premises -> conclusion
Causal	cause / mechanism / effect
Contrastive	separate competing candidates
Counterfactual	change one condition and compare
Abductive	rank explanations for an observation
Compositional / analogical	transfer a relation into another domain

Exact latent lookup from hidden-state trajectories

The recurrent state is continuous. LNGram gives successive hidden states stable discrete addresses without using surface-token boundaries.

LNGram operates on the evolving hidden-state trajectory. A learned projection produces four-bit route symbols; successive symbols form exact latent 2-gram and 3-gram addresses. Those addresses select continuous trainable latent-memory values.

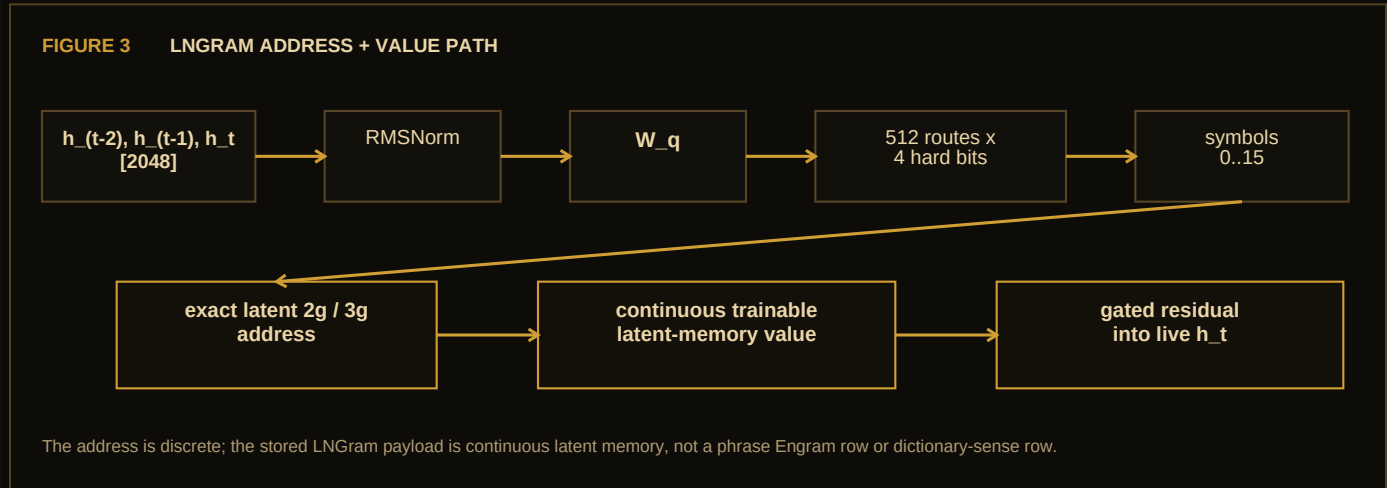


$$\text{symbol}(t,r) \text{ in } \{0,\dots,15\}$$
$$\text{addr}^n = r * 16^n + \sum_j \text{symbol}(t-n+1+j,r) * 16^j$$

ADDRESS	VALUE
deterministic for a bit sequence	continuous trainable latent value
integer lookup handle	not a phrase Engram row
order 2 uses two live states	not a dictionary sense row
order 3 uses three live states	learned from recurrent trajectories

Separation from surface memory

Phrase Engrams are already discrete exact tables, and dictionary senses already have stable row identities. LNGram exists for the continuous recurrent trajectory: it turns live hidden-state history into a sparse latent address.



Harmonic Loss turns bounded evidence into signed movement

Candidate retrieval and branch binding determine which points are present. HarMax determines how the live continuous state moves among them.

$$\begin{aligned} D_q &= ||u - c_q||^2 + \text{bounded auxiliary terms} \\ p_q &= D_q^{(-n/2)} / \sum_j D_j^{(-n/2)} \\ \rho &= - \sum_q y_q \log p_q \\ \Delta u &= n * \sum_q (y_q - p_q) * (c_q - u) / D_q \end{aligned}$$

ATTRACTION	REPULSION
$y_q > p_q$ supported anchor has too little mass movement toward c_q frozen anchor stays fixed	$y_q < p_q$ candidate has excess proximity mass movement away from c_q zero-target candidates still contrast

Harmonic residual

The harmonic residual ρ measures the mismatch between evidence target mass y and the current inverse-distance HarMax mass p . Its negative derivative gives the signed update. The live query moves; frozen coordinates do not.

What HarMax contributes

The field gives the reasoning engine a geometric reason for changing direction: the movement is tied to the signed evidence imbalance of a bounded set of explicit anchors rather than to a dense attention score.

Two physical blocks, recurrent thought depth

The recurrence is architectural; DeepLoop supplies the loop-aware residual scaling used when those blocks are revisited.

Dendritron stores two ordered physical blocks and reuses them for R rounds. Stored block depth stays 2 while the sequential unrolled thought depth is 2R.

$$\begin{aligned} H^{(r,1)} &= \text{Phi}_1(H^{(r,0)}, \text{memory}, \text{LN Gram}, \text{routing}) \\ H^{(r+1,0)} &= \text{Phi}_2(H^{(r,1)}, \text{memory}, \text{LN Gram}, \text{routing}) \end{aligned}$$

BLOCK 1 - EXPANSION retrieve / associate / propose open several plausible relations instantiate candidate branch paths carry richer state forward	BLOCK 2 - CONTRACTION contrast / verify / integrate re-evaluate proposals apply supported + opposing evidence return concentrated state to Block 1
---	--

Two sequential residual sublayers per block visit

The first sublayer performs geometric/memory/branch contraction. The second applies the selected skill LoRAs together with the expert-soma contribution. Both are Post-RMSNorm residual updates using the loop-aware skip scale alpha.

$$\begin{aligned} C_k^{(r)} &= \Delta_{\text{HarMax}} + \Delta_{\text{memory}} + \Delta_{\text{branch}} \\ U^{(r,k)} &= \text{RMSNorm}(\alpha * H_{\text{in}}^{(r,k)} + C_k^{(r)}) \\ E_k^{(r)} &= \Delta_{\text{skill-LoRA}} + \Delta_{\text{expert-soma}} \\ H_{\text{out}}^{(r,k)} &= \text{RMSNorm}(\alpha * U^{(r,k)} + E_k^{(r)}) \end{aligned}$$

Loop-aware scaling

The general tied-visit condition and Dendritron's K=2 specialization are shown once, together.

Visit accounting

$$\begin{aligned} K &= 2 \\ N &= K R = 2R \\ M &= 2N = 4R \end{aligned}$$

General tied-visit condition

$$M * \kappa_R * (\beta / \alpha)^2 = O(1)$$

κ_R measures the alignment of sensitivities across repeated visits to shared parameters. The point of the rule is to account for recurrence explicitly rather than treating the unrolled computation as an ordinary untied deep stack.

Dendritron adopts the conservative $p = 1/2$ scaling

$$\begin{aligned} \alpha &= (2N)^{1/2} = 2 * \sqrt{R} \\ \beta &= (8N)^{-1/2} = 1 / (4 * \sqrt{R}) \\ \beta / \alpha &= 1 / (4N) = 1 / (8R) \end{aligned}$$

ALPHA - RUNTIME SKIP used in every residual sublayer scales the identity path depends on configured round budget applied on every visit	BETA - INITIALIZATION GAIN applied once at initialization scales designated residual matrices not re-applied each round separate from runtime recurrence
---	--

This is a first-order perturbation/stability rule under the DeepLoop analysis, not a proof that training cannot diverge. Visit alignment and recurrent behavior still have to be measured in Dendritron.

Mechanism provenance and evidence boundary

The brief separates borrowed mechanisms, Dendritron synthesis, built assets, and proposed training work.

Source	Role
Conditional Memory via Scalable Lookup	Exact conditional-memory tables and deterministic sparse lookup.
Memory Grafting: Scaling Language Model Capacity without Retraining	Offline hidden-state extraction as the values stored in exact phrase memory.
LNGram	Hidden-state discretization into exact latent 2/3-gram addresses with continuous table values.
Harmonic Loss Trains Interpretable AI Models	HarMax inverse-distance mass and harmonic cross-entropy geometry.
DeepLoop: Depth Scaling for Looped Transformers	Visit-aware residual scaling for repeatedly reused physical blocks.
Shared LoRA Subspaces for Almost Strict Continual Learning	Shared-factor consolidation and lightweight reusable parameter structure.
Procedural Knowledge Is Not Low-Rank	Motivation to measure procedural update rank instead of assuming one uniform tiny LoRA.
Stealing Reasoning Traces from Proprietary LLM APIs	Motivation for process-rich supervision; Dendritron uses natively available or human-authored reasoning rather than extraction attacks.

Built / measured

The phrase Engram banks, frozen definition bank, token embeddings, and fitted Layer-8 / Layer-24 JTD maps are completed assets. Exact runtime integration continues to be validated against those real assets.

Proposed / requires empirical validation

Human-directed directional skill training, orthogonal operational-skill consolidation, the oblique shared LoRA, measured private residual ranks, high-dimensional expert-coordinate formation, and reverse-orthogonalization around protected experts are design/training hypotheses that must be tested rather than presented as completed results.